

ui.teleport 全面详解

一、核心定义与作用

`ui.teleport` 是 NiceGUI 框架中的核心元素，其核心功能是将组件内部的内容“传送”到页面上的任意指定位置，打破常规的 DOM 层级嵌套限制。无论原始组件的结构如何，通过 `ui.teleport` 可将目标内容精准注入到目标元素内部或指定 DOM 节点，实现灵活的内容布局与嵌套扩展。

核心价值：解决常规布局中“内容结构与渲染位置强绑定”的问题，支持在不修改原始 DOM 层级的前提下，将复杂内容（如输入框、图标、图表等）嵌入到目标元素（如表格单元格、单选框标签、markdown 文本节点等）中。

二、核心参数

`ui.teleport` 的初始化仅需一个核心参数，决定内容的传送目标：

参数	类型	说明
<code>to</code>	NiceGUI 元素 或 CSS 选择器	接收传送内容的目标对象。可以是 NiceGUI 已创建的元素实例（如 <code>table</code> 、 <code>radio</code> ），也可以是 CSS 选择器字符串（如 <code>#id > div:nth-child(2)</code> ），用于精准定位 DOM 节点

三、关键属性

`ui.teleport` 继承自 NiceGUI 基础元素，拥有以下核心属性（用于元素样式、状态控制等）：

属性名	类型	说明	
<code>classes</code>	<code>Classes[Self]</code>	元素的 CSS 类名，支持 Tailwind/Quasar 样式类，用于自定义外观	
<code>client</code>	<code>Client</code>	该元素所属的客户端实例，用于多客户端场景下的隔离	
<code>html_id</code>	<code>str</code>	元素在 HTML DOM 中的唯一 ID（2.16.0 版本新增），可用于 CSS 选择器定位	
<code>is_deleted</code>	<code>bool</code>	元素是否已被删除，用于状态判断	
<code>is_ignoring_events</code>	<code>bool</code>	是否忽略事件触发，用于临时禁用交互	
<code>parent_slot</code>	<code>`Slot`</code>	<code>None`</code>	元素的父级插槽，支持修改（NiceGUI 基于 Vue 插槽机制）

属性名	类型	说明	
<code>props</code>	<code>Props[Self]</code>	元素的 Quasar 组件属性，以 HTML 属性形式生效	
<code>style</code>	<code>Style[Self]</code>	元素的内联 CSS 样式，用于精细化样式控制	
<code>visible</code>	<code>BindableProperty</code>	元素的可见性，支持双向绑定（可通过 <code>bind_visibility</code> 关联其他对象属性）	

四、核心方法

`ui.teleport` 提供丰富的方法用于元素控制、事件绑定、资源管理等，以下是常用核心方法：

1. 可见性绑定相关

- `bind_visibility(target_object, target_name='visible', ...)`：双向绑定元素可见性到目标对象的属性。例如将 `teleport` 可见性与某个开关组件的 `value` 绑定，实现联动显示 / 隐藏。
- `bind_visibility_from(target_object, ...)`：单向绑定（从目标对象到当前元素），仅同步目标对象属性变化到元素可见性。
- `bind_visibility_to(target_object, ...)`：单向绑定（从当前元素到目标对象），仅同步元素可见性变化到目标对象属性。
- `set_visibility(visible: bool)`：直接设置元素可见性（显式控制显示 / 隐藏）。

2. 元素操作相关

- `clear()`：移除所有子元素（清空传送的内容）。
- `delete()`：删除当前 `teleport` 元素及所有子元素，释放资源。
- `move(target_container, target_index=-1, target_slot=None)`：将 `teleport` 元素移动到其他容器中，支持指定目标插槽和索引位置。
- `remove(element: Element | int)`：移除指定子元素（传入元素实例或其 ID）。
- `update()`：触发客户端同步，更新元素状态（如样式、内容变化后手动刷新）。

3. 资源与插槽管理

- `add_resource(path: str | Path)`：为元素添加资源（如 CSS/JS 文件目录），用于加载自定义样式或脚本。
- `add_dynamic_resource(name: str, function: Callable)`：添加动态资源，通过函数返回资源响应（支持动态生成内容）。
- `add_slot(name: str, template: str | None = None)`：为元素添加 Vue 插槽，用于复杂内容嵌套（NiceGUI 基于 Vue 插槽机制，支持多插槽布局）。

4. 事件与交互相关

- `on(type: str, handler, ...)`: 绑定事件处理器。支持 Python 函数（服务端处理）、JavaScript 函数（客户端处理），或两者结合。例如绑定 `click` 事件触发内容更新。
 - 参数说明: `type` 为事件名（如 `click`、`update:model-value`），`handler` 为 Python 回调函数，`js_handler` 为客户端 JS 函数，`throttle` 用于事件节流控制。
- `tooltip(text: str)`: 为元素添加 tooltip 提示（鼠标悬浮时显示文本）。

5. 查询与遍历相关

- `ancestors(include_self: bool = False)`: 迭代遍历元素的所有祖先节点（可选包含自身）。
- `descendants(include_self: bool = False)`: 迭代遍历元素的所有后代节点（可选包含自身）。
- `get_computed_prop(prop_name: str, timeout: float = 1)`: 异步获取元素的计算属性（需 await 等待结果）。
- `mark(*markers: str)`: 为元素添加标记，用于测试查询或依赖管理（替换现有标记）。
- `run_method(name: str, *args: Any, timeout: float = 1)`: 在客户端执行元素方法（支持传参，可选等待响应）。

6. 样式与属性默认配置

- `default_classes(add/remove/toggle/replace)`: 为所有该类元素设置默认 CSS 类（需在实例化前调用）。
- `default_props(add/remove)`: 为所有该类元素设置默认 Quasar 属性（需在实例化前调用）。
- `default_style(add/remove/replace)`: 为所有该类元素设置默认内联样式（需在实例化前调用）。

五、实战示例

示例 1：向 Markdown 文本中注入输入框

功能：点击按钮后，将输入框注入到 Markdown 文本的 `<strong>` 标签内（即“name”加粗文本位置）。

```
from nicegui import ui

# 创建 Markdown 元素，内容包含加粗文本“name”
markdown = ui.markdown('Enter your name!')

def inject_input():
    # 传送输入框到 Markdown 元素的 <strong> 标签内
    with ui.teleport(f'#{markdown.html_id} strong'):
        ui.input('name').classes('inline-flex').props('dense outlined')

# 触发注入的按钮
ui.button('inject input', on_click=inject_input)

ui.run()
```

示例 2：为单选框添加图标内容

功能：自定义单选框选项，将图标注入到单选框的标签位置，替代默认文本。

```
from nicegui import ui

options = ['Star', 'Thump Up', 'Heart']
# 创建单选框（选项值为空字符串，后续用图标替换）
radio = ui.radio({x: ' for x in options}, value='Star').props('inline')

# 向第 1 个单选框标签注入“星星”图标
with ui.teleport(f'#{radio.html_id} > div:nth-child(1) .q-radio__label'):
    ui.icon('star', size='md')
# 向第 2 个单选框标签注入“点赞”图标
with ui.teleport(f'#{radio.html_id} > div:nth-child(2) .q-radio__label'):
    ui.icon('thumb_up', size='md')
# 向第 3 个单选框标签注入“爱心”图标
with ui.teleport(f'#{radio.html_id} > div:nth-child(3) .q-radio__label'):
    ui.icon('favorite', size='md')

# 显示当前选中的选项值
ui.label().bind_text_from(radio, 'value')

ui.run()
```

示例 3：向表格单元格注入 ECharts 图表

功能：在表格的“Sales”列单元格中，注入折线图展示详细数据。

```
from nicegui import ui

# 表格列定义（Product 列显示产品名，Sales 列显示图表）
columns = [
    {'name': 'name', 'label': 'Product', 'field': 'name', 'align': 'center'},
    {'name': 'sales', 'label': 'Sales', 'field': 'sales', 'align': 'center'},
]
# 表格行数据（包含产品名和对应的销售数据）
rows = [
    {'name': 'A', 'data': [10, 8, 2, 4]},
    {'name': 'B', 'data': [3, 5, 7, 8]},
    {'name': 'C', 'data': [2, 1, 3, 7]},
]

# 创建表格
table = ui.table(columns=columns, rows=rows, row_key='name').classes('w-72')

# 遍历行数据，向每个行的第 2 个单元格（Sales 列）注入折线图
for r, row in enumerate(rows):
    with ui.teleport(f'#{table.html_id} tr:nth-child({r+1}) td:nth-child(2)'):
        ui.echart({
            'xAxis': {'type': 'category', 'show': False}, 隐藏 X 轴
            'yAxis': {'type': 'value', 'show': False}, 隐藏 Y 轴
```

```
'series': [{ 'type': 'line', 'data': row['data'] }], 折线图数据  
}).classes('w-44 h-20') 图表尺寸
```

```
ui.run()
```

六、使用场景总结

1. **自定义组件内容**：向 NiceGUI 内置组件（如单选框、表格、卡片）中注入自定义内容（图标、输入框、图表等），扩展组件功能。
2. **跨层级 DOM 操作**：无需修改原始组件结构，即可将内容插入到目标 DOM 节点，简化复杂布局实现。
3. **动态内容注入**：结合事件触发（如按钮点击），动态向目标位置添加 / 移除内容，实现交互性布局。
4. **样式与结构分离**：保持原始组件的逻辑结构，仅通过 `teleport` 调整渲染位置，提升代码可维护性。

七、注意事项

1. **目标定位准确性**：使用 CSS 选择器时，需确保目标 DOM 节点的选择器唯一且正确（可通过浏览器开发者工具查看 DOM 结构）。
2. **元素生命周期**：`teleport` 传送的内容会随目标元素的删除而被删除，需注意父子元素的生命周期关联。
3. **版本兼容性**：`html_id` 属性从 2.16.0 版本开始支持，`toggle` 参数在 `default_classes` 中从 2.7.0 版本开始支持，使用时需注意 NiceGUI 版本。
4. **性能考虑**：频繁动态创建 / 删除 `teleport` 元素时，建议配合 `delete()` 方法释放资源，避免内存泄漏。